

Secure Software Development – GRC Portal Project

Author: Saroj Kushwaha

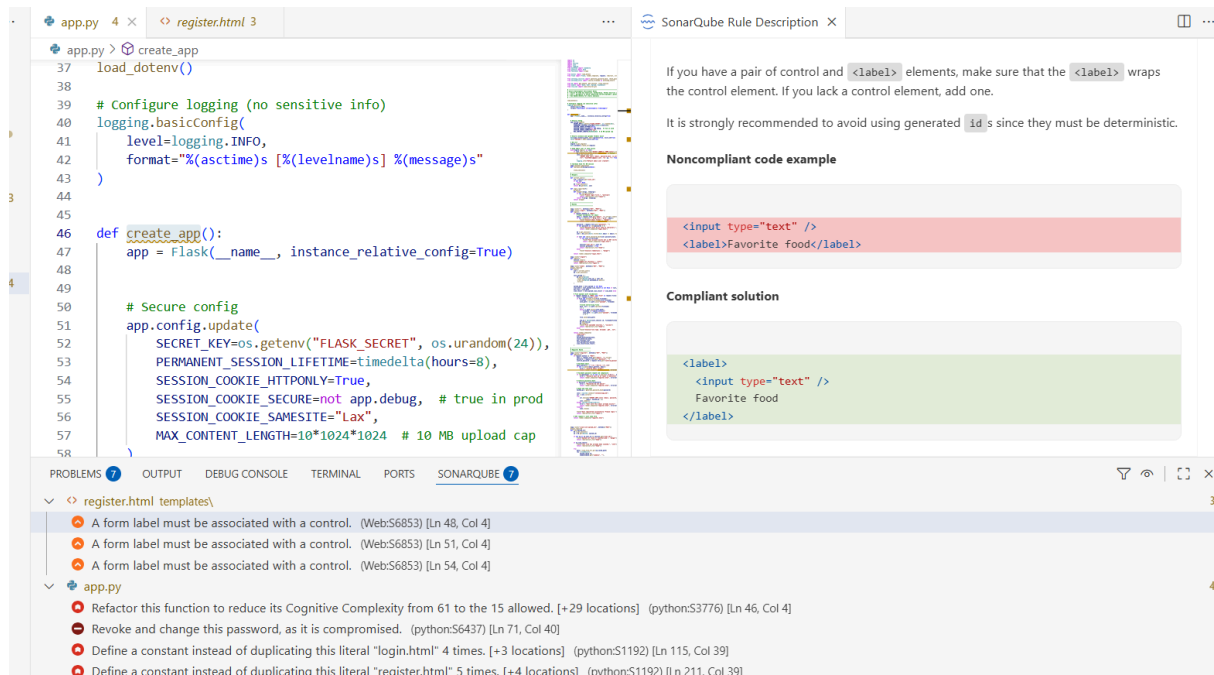
Date: 28 Aug 2025

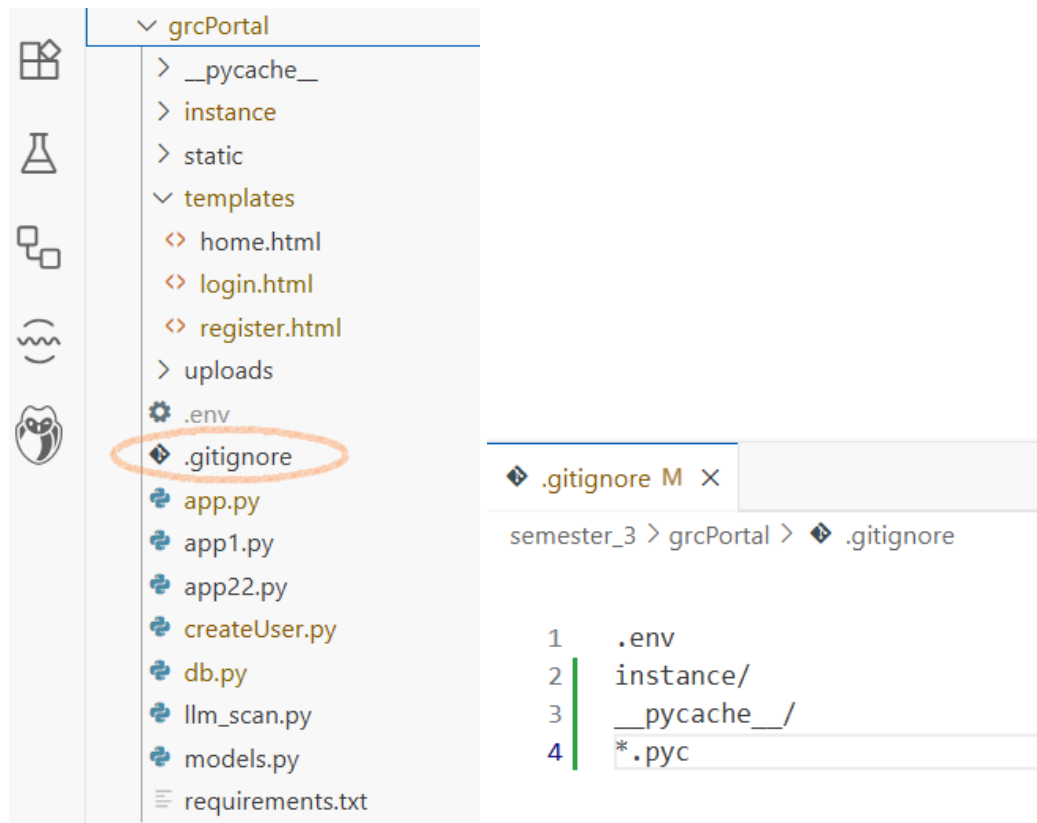
1. Set Up a Secure Development Environment

To ensure a secure development environment, the following steps were implemented:

- Installed and configured VS Code extensions: SonarLint, GitGuardian, and Bandit.
- Verified that GitGuardian detects secret leaks (tested with a fake API key).
- Configured `.gitignore` to exclude sensitive files such as `.env`, `__pycache__`, `instance/`, and database files.
- Demonstrated secure Git configuration using SSH keys.
- Secured basic web framework settings (Flask) with `SECRET_KEY` from `.env` and secure session cookie configuration.

Screenshots of extensions in VS Code and `.gitignore` configuration are attached as evidence.





2. Implement Secure Coding Practices

The application implements input validation, output encoding, secure error handling, and authentication as required:

- **Input Validation**: Email addresses validated with regex, password rules enforced (min 8 characters, must include letters, numbers, special character). File uploads restricted to safe types (`.pdf`, `.txt`).
- **Output Encoding**: All displayed user input is escaped using Jinja2 autoescaping (prevents XSS).
- **Secure Error Handling**: Implemented custom 404 and 500 error pages. Stack traces are logged internally but not displayed to users.
- **Authentication**: User credentials stored securely using salted + hashed passwords (generate_password_hash` and `check_password_hash`).

Code examples and screenshots demonstrate these implementations.

```
# Validate email
email_pattern = r'^[\w\.-]+@[\w\.-]+\.\w+$'
if not re.match(email_pattern, email):
    error = "Invalid email format!"
    return render_template("register.html", error=error)
```

```

# Validate password length and complexity
if len(password) < 8 or not re.search(r'[A-Za-z]', password) or not re.search(r'\d', password):
    error = "Password must be at least 8 characters long and contain at least one alphabet and one number."
    return render_template("register.html", error=error)

# Confirm password check
if password != confirm_password:
    error = "Passwords do not match!"
    return render_template("register.html", error=error)

# Hash and save user
hashed_pw = generate_password_hash(password)

conn = sqlite3.connect("instance/app.db")
cur = conn.cursor()
try:
    cur.execute("INSERT INTO users (email, password_hash, is_verified) VALUES (?, ?, ?)",
                (email, hashed_pw, 1))
    conn.commit()
except sqlite3.IntegrityError:
    error = "User with this email already exists!"
    return render_template("register.html", error=error)
finally:
    conn.close()

# error handlers
@app.errorhandler(404)
def not_found(e):
    return render_template("errors/404.html"), 404

@app.errorhandler(500)
def server_error(e):
    logging.error(f"500 Error: {e}")
    return render_template("errors/500.html"), 500

```

3. Perform Code Analysis and Security Testing

Security testing was carried out using static analysis tools, code review, and automated test cases:

- **Static Analysis**: Bandit was run on the repository, with HTML and JSON reports generated. SonarLint and GitGuardian extensions detected and prevented issues in VS Code.
- **Code Review**: A checklist-based review was conducted for authentication, input validation, error handling, and secure configurations.
- **Security Test Cases**: At least 6 test cases were implemented (positive, negative, edge cases):
 - Valid registration/login (positive).
 - Invalid email, weak password, password mismatch (negative).
 - Minimum password length and path traversal upload attempts (edge cases).
- Test results were documented in logs (`pytest.log`) and stored in the reports folder.

```
PS C:\Users\saroj\OneDrive\Desktop\course\FirstSetup\semester_3\grcPortal> mkdir -p reports
>> bandit -r . -f html -o reports/bandit.html
>> bandit -r . -f json -o reports/bandit.json
```

Directory: C:\Users\saroj\OneDrive\Desktop\course\FirstSetup\semester_3\grcPortal

Mode		LastWriteTime	Length	Name
d----		8/28/2025 12:44 PM		reports
[main]	INFO	profile include tests: None		
[main]	INFO	profile exclude tests: None		
[main]	INFO	cli include tests: None		
[main]	INFO	cli exclude tests: None		
[main]	INFO	running on Python 3.13.5		
[html]	INFO	HTML output written to file: reports/bandit.html		
[main]	INFO	profile include tests: None		
[main]	INFO	profile exclude tests: None		
[main]	INFO	cli include tests: None		
[main]	INFO	cli exclude tests: None		
[json]	INFO	JSON output written to file: reports/bandit.json		

```
PS C:\Users\saroj\OneDrive\Desktop\course\FirstSetup\semester_3\grcPortal> 
```

Metrics:

Total lines of code: 695

Total lines skipped (#nosec): 0

flask_debug_true: A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
Test ID: B201
Severity: HIGH
Confidence: MEDIUM
CWE: [CWE-94](#)
File: [app1.py](#)
Line number: 204
More info: https://bandit.readthedocs.io/en/1.8.6/plugins/b201_flask_debug_true.html

```
203     app = create_app()
204     app.run(debug=True)
```

flask_debug_true: A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
Test ID: B201
Severity: HIGH
Confidence: MEDIUM
CWE: [CWE-94](#)
File: [app22.py](#)
Line number: 215
More info: https://bandit.readthedocs.io/en/1.8.6/plugins/b201_flask_debug_true.html

```
214     app = create_app()
215     app.run(debug=True)
```

The screenshot shows a VS Code editor with a Python file named 'app.py' open. The code includes logging configuration and a Flask application setup. A SonarQube rule description is displayed on the right, explaining the issue with form labels and controls. The rule states that if a pair of control and <label> elements are used, the <label> must wrap the control element. It also notes that generated IDs are not deterministic. A noncompliant code example is shown: <input type="text" /> <label>Favorite food</label>. A compliant solution is also provided: <label> <input type="text" /> Favorite food </label>.

Problems:

- register.html templates\
 - A form label must be associated with a control. (WebS6853) [Ln 48, Col 4]
 - A form label must be associated with a control. (WebS6853) [Ln 51, Col 4]
 - A form label must be associated with a control. (WebS6853) [Ln 54, Col 4]
- app.py
 - Refactor this function to reduce its Cognitive Complexity from 61 to the 15 allowed. [+29 locations] (pythonS3776) [Ln 46, Col 4]
 - Revoke and change this password, as it is compromised. (pythonS6437) [Ln 71, Col 40]
 - Define a constant instead of duplicating this literal "login.html" 4 times. [+3 locations] (pythonS1192) [Ln 115, Col 39]
 - Define a constant instead of duplicating this literal "register.html" 5 times. [+4 locations] (pythonS1192) [Ln 211, Col 39]

4. Conduct a Basic Vulnerability Assessment

Two vulnerabilities from OWASP Top 10 were tested and documented:

1. **SQL Injection (A03:2021 – Injection)**

- Steps: Attempted `` OR '1'='1` in login input.
- Result: Login was blocked (parameterized queries prevented injection).
- Remediation: Continued use of parameterized queries.

GRC Portal

Email

Password

 Please include an '@' in the email address. `` OR '1'='1' is missing an '@'.

Login

Don't have an account? [Create new user](#)

GRC Portal

Email

Password

 A part followed by '@' should not contain the symbol ' '.

Login

Don't have an account? [Create new user](#)

2. **Weak Password Policy (A07:2021 – Authentication Failures)**

- Steps: Attempted registration with password `12345678`.
- Result: Registration blocked due to regex validation.
- Remediation: Enforced strong password rules (length, upper/lowercase, number, special character).

GRC Portal

Invalid credentials.

Email

Password

Login

5. Apply Security Best Practices in Development

Security best practices applied in the project include:

- **Coding Standards**: Used parameterized queries instead of string concatenation. All passwords are hashed before storage.
- **Input Validation & Sanitization**: Regex validation for emails, strong password enforcement, safe file upload validation.
- **Output Encoding**: Jinja2 autoescaping ensures user-supplied data is encoded.
- **Error Handling**: Generic user error messages; sensitive errors logged securely.
- **Configuration Hardening**:
 - Secrets loaded from `.env` file, not hardcoded.
 - `.gitignore` prevents accidental commits of sensitive files.
 - Flask `debug=False` in production.
 - Session cookies configured with `HttpOnly`, `Secure`, `SameSite=Lax`.

Screenshots and configuration examples are included as evidence.

```
# Confirm password check
if password != confirm_password:
    error = "Passwords do not match!"
    return render_template("register.html", error=error)

# Hash and save user
hashed_pw = generate_password_hash(password)

conn = sqlite3.connect("instance/app.db")
cur = conn.cursor()
try:
    conn.exec_driver_sql(
        "INSERT INTO users (email, password_hash, is_verified) VALUES (:e, :p, :v)",
        {"e": "kush786srj@gmail.com", "p": pw, "v": True},
    )
```

In the above code “{.....}” allows vulnerability attacks and insert any value which could not be safe for the organization.

The safe way is to use “(....)” as shown below to avoid any vulnerability attack

```
try:
    cur.execute("INSERT INTO users (email, password_hash, is_verified) VALUES (?, ?, ?)",
                (email, hashed_pw, 1))
    conn.commit()
except sqlite3.IntegrityError:

# Secure config
app.config.update(
    SECRET_KEY=os.getenv("FLASK_SECRET", os.urandom(24)),
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SECURE=not app.debug, # true in prod
    SESSION_COOKIE_SAMESITE="Lax",
    MAX_CONTENT_LENGTH=10*1024*1024 # 10 MB upload cap
)
```

6. Produce Security Documentation and Reporting

Comprehensive reporting was produced to document all security measures:

- **Security Issue Report**: Table of issues with severity, evidence, remediation, and status.
- **Test Case Documentation**: Includes inputs, expected results, actual results, and pass/fail.
- **Vulnerability Assessment Report**: Detailed analysis of SQL Injection and Weak Password vulnerabilities with steps to reproduce and remediation.
- **Evidence**: Screenshots of tool usage (SonarLint, GitGuardian, Bandit), Flask error pages, test results, and database records (hashed passwords).